

EPV: An Assurance-Oriented Framework for Verifying and Synthesizing Finite Economic Protocols

Aayush Kadam

Abstract

Economic protocols combine executable rules with behavioral and allocation claims that are easy to state informally and easy to misinterpret in software. EPV is a research-preview framework for bounded verification of selected finite auctions, one-to-one matching markets, and indivisible-goods fair-division instances. It provides typed exact semantics, exhaustive and SMT-backed property checking, replayable exactly minimized counterexamples, independently checked Alethe certificates for compiled finite propositional obligations, and exhaustive synthesis or repair over small declared auction classes. The framework treats scope and trust as first-class outputs: every claim identifies its finite domain, assumptions, property version, mechanism/problem hash, assurance level, and residual trusted computing base. A hostile validation campaign generated 120 instances and 584 property evaluations, executed 1,168 solver runs with no unresolved disagreement, killed 18 of 18 targeted semantic mutants, and found seven defects, including a certificate-integrity defect in clean archives; all critical and high-severity findings were resolved. EPV does not provide unbounded theorems, a verified economic compiler, or scalable mechanism synthesis. Its contribution is primarily assurance-oriented systems integration and an adversarial validation methodology for making bounded economic verification artifacts inspectable.

1. Introduction

Mechanism descriptions mix economic assumptions, behavioral quantifiers, allocation rules, transfer conventions, and implementation details. A simulation can expose one bad execution but cannot establish the absence of a violation over even a modest finite domain unless coverage is explicit. A solver can report unsatisfiable, but that answer is meaningful only if the economic property was translated correctly and the result is bound to the intended mechanism, assumptions, and formula.

EPV addresses this narrower problem: given a supported deterministic finite economic protocol and an explicit property, construct every relevant violation case, search it through reference and solver backends, and emit evidence whose scope can be inspected and replayed. The system supports deterministic single-item auctions, strict one-to-one matching, and additive indivisible-goods fair division. A separate finite search layer synthesizes and repairs tiny auction tables.

EPV does not prove arbitrary markets, continuous or randomized mechanisms, Bayesian properties, or general economic theorems. Its positive certificates establish only that an independently implemented checker accepted the compiled finite obligation. The economic-semantics-to-obligation translation remains trusted.

The artifact makes four claims. First, one typed workflow can express useful bounded checks across three institution families without erasing domain-specific semantics. Second, property failures can be carried as typed, hash-bound, exactly minimized witnesses and replayed independently of solver expressions. Third, positive results for a deliberately limited propositional fragment can carry Alethe proofs freshly checked by pinned Carcara. Fourth, finite synthesis and repair results can use the same property evaluators, explicit class bounds, and post-verification. These are systems and methodological contributions; none is presented as a new economic theorem or proof technology.

2. Motivation and scope

Ordinary unit tests sample executions chosen by developers. Economic properties such as dominant-strategy incentive compatibility quantify over agents, true types, deviations, and other reports. Matching stability quantifies over potential blocking pairs. EF1 quantifies over ordered agent pairs and goods in an envied bundle. In finite instances these obligations can be enumerated exactly, yielding bounded guarantees and concrete failure witnesses.

Bounded verification is valuable for executable examples, regression testing, protocol pedagogy, and falsification. It is not a replacement for symbolic theorem proving. The measured EPV envelope reaches selected DSIC checks at four bidders with two values, a canonical 2x2 matching market, small additive fair-division instances, and auction synthesis spaces up to 81 measured candidates.

3. Related work

Automated mechanism design dates at least to Conitzer and Sandholm's early complexity work. Formal auction verification includes Isabelle/HOL work by Caminati and collaborators, computer-aided incentive proofs by Barthe and collaborators, and the Coq/Mathematical Components `mech.v` library. Strategy-Logic work by Mittelmann et al. directly integrates verification and synthesis for social-choice mechanisms. SAT/SMT reasoning has long supported countermodels, impossibility results, and minimal inconsistent sets in computational social choice; Brandl et al. connect SMT evidence to Isabelle/HOL reconstruction.

EPV therefore does not claim priority for mechanism verification, synthesis, SMT use, machine-checked auction results, or proof reconstruction. Alethe and Carcara also pre-exist EPV. The difference is operational: EPV combines a small readable source form, exact executable economic semantics, property-to-failure compilation, positive and negative artifacts, multiple backends, finite synthesis, and hostile artifact validation under one explicit assurance policy. Whether this combination constitutes a publishable research contribution requires external comparison, especially with the Strategy-Logic system.

4. Economic protocol model

An auction mechanism maps reports to an allocation and a vector of transfers. Reports are distinct typed values from true types even when their carrier sets coincide. Transfers are signed as money paid by the agent. Utility is quasi-linear, $u_i(\theta_i, x, p) = v_i(\theta_i, x) - p_i$. Deterministic tie breaking and outside options are explicit.

Matching uses strict complete ordinal rankings over the opposite side plus unmatched. Comparisons use rank order only; no cardinal utility is invented. Fair division assigns indivisible goods under additive nonnegative integer valuations. Each domain has its own semantic hash namespace and property registry.

EPL 0.1 is a JSON-based auction source language with bounded integer types/reports and four allocation/payment templates. Parsing rejects duplicate keys, floating-point literals, ambiguous tie breaking, wrong transfer signs, unknown constructs, and oversized sources/domains.

5. Verification architecture

The pipeline is source or domain object, typed semantic representation, exact evaluator, violation-case compiler, and one or more backends. V1 evaluates every declared case. V2 encodes the neutral witness problem separately for cvc5 and Z3; SAT means a violation exists and UNSAT means none of the materialized violation predicates holds. Cross-backend agreement can detect adapter defects but cannot detect a shared evaluator error.

Every property has a versioned identifier, applicability conditions, assumptions, verification method, maximum demonstrated assurance, and limitations. Cross-domain property requests fail rather than silently reinterpret a property.

6. Counterexamples

A failed property yields a typed certificate containing mechanism and logical-problem hashes, property/version, domain, assumptions, backend origin, exact witness, replay result, and minimality metadata. The minimizer enumerates all replayable violations and selects a deterministic lexicographic minimum: number of nonzero numeric fields, absolute magnitude, denominator sum, then canonical JSON.

Replay recomputes the minimum through the authoritative evaluator. It does not trust the solver's printed model or the human-readable renderer. This is backend-independent negative evidence but not semantics-independent evidence because compilation and replay share economic semantics.

7. Proof-carrying positive verification

For selected small properties, EPV materializes a Boolean proposition for every exact violation case and asserts that at least one violation exists. When every proposition is false, `cvc5` returns UNSAT and emits Alethe. A manifest binds exact problem and proof bytes to the mechanism/semantic hash, property, domain, assumptions, producer, checker identity, and EPV commit. Pinned Carcara must freshly output `valid`.

This is V3 bounded independently certificate-checked verification. Carcara checks the compiled logical refutation, not the correctness of the economic evaluator or compiler. The current path uses propositional truth tables because direct arithmetic Alethe proofs tested during development contained unsupported holes. Carcara is independently implemented but not a formally verified theorem-prover kernel.

8. Cross-domain semantics

Auctions support DSIC, ex-post individual rationality, weak and strong budget balance, feasibility, and allocative efficiency. Matching supports feasibility, individual rationality, stability, and proposer strategyproofness in the declared report universe. Fair division supports feasibility, envy-freeness, EF1, and Pareto efficiency.

Representative positive cases include a second-price auction, proposer deferred acceptance, and round robin for EF1. Representative negative cases include first-price DSIC, a fixed unstable matching, and a round-robin envy failure. V3 bundles cover auction properties, matching stability, fair-division EF1, and one synthesis bounded-UNSAT obligation.

9. Finite synthesis and repair

Auction synthesis enumerates two-bidder tables with values $\{0, 1\}$, declared payment values, and either a fixed efficient allocation or all feasible allocations. Hard requirements use the normal property evaluators. Optional expected revenue or welfare uses an explicit exact prior. Candidates are post-verified; objective ties use canonical serialization.

Payment repair preserves an efficient allocation table and minimizes changed cells, total absolute adjustment, then lexical order. Class membership is checked before optimization. Integer reserve synthesis enumerates an explicit grid. An UNSAT result means only that no candidate exists in the declared finite class. Exhaustive enumeration establishes current optimum/minimum claims; it does not scale as a synthesis algorithm.

10. Evaluation

The M7 adversarial campaign used seed 7001 and generated 64 auction, 24 matching, and 32 fair-division instances. Across 584 property evaluations, both `cvc5` and Z3 executed 584 runs; each returned 292 SAT and 292 UNSAT, with no unknown, timeout, error, or unresolved discrepancy. The campaign killed 18 of 18 targeted semantic mutants, exercised 200 malformed parser inputs plus six valid formatting variants, ran 19 certificate attacks, and compared 32 synthesis problems with independent exhaustion. No false verified, counterexample, V3, optimal, or bounded-UNSAT result remained in the tested fragments.

Scaling measurements are intentionally small. Auction DSIC contained 16 cases for two bidders/two values, 54 for two/three, 48 for three/two, and 128 for four/two. On the recorded machine, reference compilation ranged from 1.819 to 15.105 ms, `cvc5` from 6.510 to 10.544 ms except per-run startup effects, and Z3 from 16.154 to 29.478 ms. Synthesis of 1, 16, and 81 candidates measured 3.884, 13.390, and 52.215 ms. These values characterize one machine and must not be extrapolated.

11. Adversarial validation

Hostile review found seven material defects. A critical Windows line-ending issue changed proof/formula bytes in a clean Git archive and invalidated hashes; proof artifacts are now marked binary. High-severity issues involved missing proof files not failing cleanly, malformed optimization priors, and repair of a baseline outside the declared allocation class. Medium findings covered a unary `cvc5` disjunction, parser resource limits, and cold WSL checker timeout. Each finding has a regression test. The exercise demonstrates why release-level checks belong inside the scientific method for proof-carrying systems.

12. Trusted computing base

V1 trusts parser/elaborator, semantic objects, evaluator, predicates, exact arithmetic, runtime, and platform. V2 adds translation and solver trust. V3 removes reliance on cvc5's UNSAT answer for the exact checked formula but retains the economic translation and Carcara in the trusted base. SHA-256 and canonical serialization bind context and bytes; they do not validate user intent or compiler correctness. Synthesis adds the search compiler, enumerator, objective, membership checks, and status interpreter.

13. Limitations

All verification is finite and bounded. EPL is template-based. There is no voting, randomized or Bayesian mechanism support, continuous space, symbolic unbounded proof, many-to-one matching, ties, non-additive fair division, or implementation-conformance proof. Synthesis spaces are tiny and auction-only. The semantic translation is not verified. Carcara depends on a pinned WSL setup and is not formally verified. Proofs are small truth-table certificates. Mutation analysis is targeted, not exhaustive. Scaling evidence is limited and machine-specific. External user testing consists of scripted fresh-user stories, not an independent study.

14. Discussion

EPV's strongest idea is not that a solver can check a finite mechanism. It is that assurance levels should disclose which link was checked, that failures and successes should both travel with replayable evidence, and that synthesis should expose its class and post-verification. The hostile campaign also shows that provenance infrastructure is part of correctness: mathematically valid proof bytes are useless if release tooling silently changes them.

The present artifact is best evaluated as a systems/demo, artifact, or software paper. A theorem-focused formal-methods paper would require a verified or reconstructed semantic translation; an economics-theory paper would require new economic results; a synthesis paper would require materially larger or more expressive classes.

15. Conclusion

EPV demonstrates a restrained assurance workflow for finite economic protocols across three domains. It can establish bounded results, produce replayable counterexamples, independently check selected compiled proof obligations, and search tiny mechanism classes. Its claims stop at the declared bounds and trusted translation. The release candidate is ready for skeptical human review, not yet for publication.

References

- Conitzer, V., and Sandholm, T. Complexity of Mechanism Design. UAI, 2002.
- Caminati, M. B., Kerber, M., Lange, C., and Rowat, C. Sound Auction Specification and Implementation. 2015.
- Barthe, G., Gaboardi, M., Gregoire, B., Hsu, J., and Strub, P.-Y. Computer-aided Verification in Mechanism Design. 2015.
- Jouvelot, P., and Gallego Arias, E. J. A Foundational Framework for the Specification and Verification of Mechanism Design. 2021.
- Brandl, F., Brandt, F., Eberl, M., and Geist, C. Proving the Incompatibility of Efficiency and Strategyproofness via SMT Solving. JACM, 2018.
- Mittelmann, M., Maubert, B., Murano, A., and Perrussel, L. Formal verification and synthesis of mechanisms for social choice. Artificial Intelligence 339, 2025.
- Andreotti, B. et al. Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. TACAS, 2023.